WEEK 7

Exploratory Data Analysis II

Advanced visualization for pattern discovery

Day 31	**Seaborn Visualization**	histplot, kdeplot, boxplot, countplot, figure vs axes-level
Day 32	**Relationship Plots**	scatterplot, relplot, hue/style/size encoding, overplotting
Day 33	**Heatmaps & Correlations**	Correlation matrices, feature clusters, multicollinearity
Day 34	**Pattern Discovery**	Faceting, grouped plots, trend overlays, visual clusters
Day 35	**EDA Mini-Project**	Full visual story with ML recommendations

SEABORN VISUALIZATION PATTERN

```
import seaborn as sns; sns.set(style="whitegrid")
sns.histplot(df, x="feature", hue="category", kde=True)  # Distribution by group
sns.relplot(data=df, x="x", y="y", col="segment", hue="priority")  # Faceted relationships
```

DAY 31

Visualization with Seaborn

High-Level Statistical Plotting for EDA

OBJECTIVES

- Understand why Seaborn is effective for statistical plots
- Learn figure-level vs axes-level functions
- Choose appropriate plots for numeric/categorical variables
- Recreate Matplotlib plots using Seaborn
- Apply consistent aesthetics and themes

ACTIVITY

Rebuild core EDA plots using Seaborn.

Apply consistent style via sns.set().

ASSESSMENT

Notebook with Seaborn visualizations and interpretation bullets.

Why Seaborn over Matplotlib?

Higher-level, statistically aware plotting

Seaborn Advantages

- Statistically aware functions (histplot, boxplot)
- Integrated aesthetics (themes, palettes) with minimal config
- Easy pandas DataFrame integration via data= + column names
- Rapid exploratory visualizations

When to Use Matplotlib

- Highly customized, non-standard layouts
- Fine-grained control over ticks, axes, annotations
- Custom artists and complex multi-panel figures

SEABORN HISTPLOT VS MATPLOTLIB

```
sns.histplot(df["age"], bins=30, kde=False)  # One line with nice defaults
plt.title("Age Distribution")
# Matplotlib equivalent requires more setup (fig, ax, labels, etc.)
```

Figure-Level vs Axes-Level Functions

Two interfaces for different use cases

Axes-Level Functions

- histplot, kdeplot, boxplot, scatterplot
- Draw on a single Matplotlib Axes
- Controlled via standard plt.subplots(), ax=
- Best for: single precise plots, custom layouts

Figure-Level Functions

- relplot, displot, catplot, lmplot
- Create FacetGrid managing subplots and facets
- Support row/col/hue for systematic comparison
- Best for: multi-plot layouts, faceting across categories

AXES VS FIGURE LEVEL

```
fig, ax = plt.subplots(); sns.histplot(df, x="age", ax=ax)  # Axes-level
g = sns.displot(df, x="age", col="segment", height=3)     # Figure-level facets
```

histplot — Univariate Distribution

Histogram with optional KDE overlay

What histplot Shows

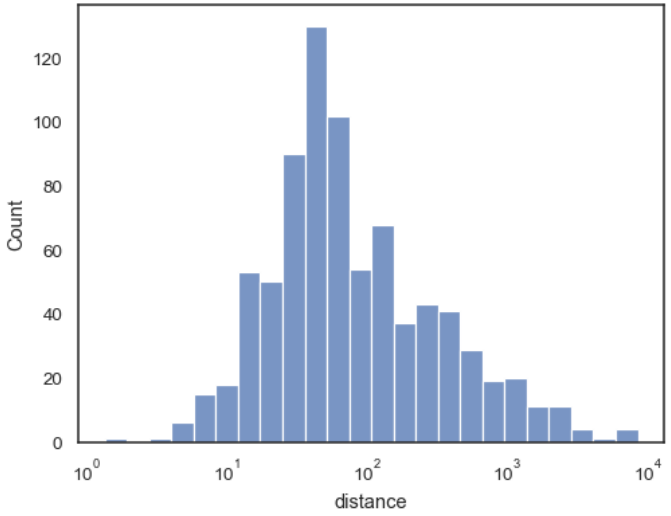
- Histogram of numeric variable with flexible binning
- Optional KDE overlay for smoothed shape
- Combines Matplotlib histogram with nicer defaults

ML Implications

- Right-skewed → consider log transformation
- Heavy tails → robust models or capping
- Bimodality → multiple customer types

HISTPLOT WITH KDE

```
plt.figure(figsize=(6, 4))
sns.histplot(df, x="income", bins=40, kde=True)
plt.title("Income Distribution — histplot with KDE")
plt.tight_layout(); plt.show()
```



histplot with Hue — Group Comparison

Compare distributions across categories in one figure

What Multi-Hue Shows

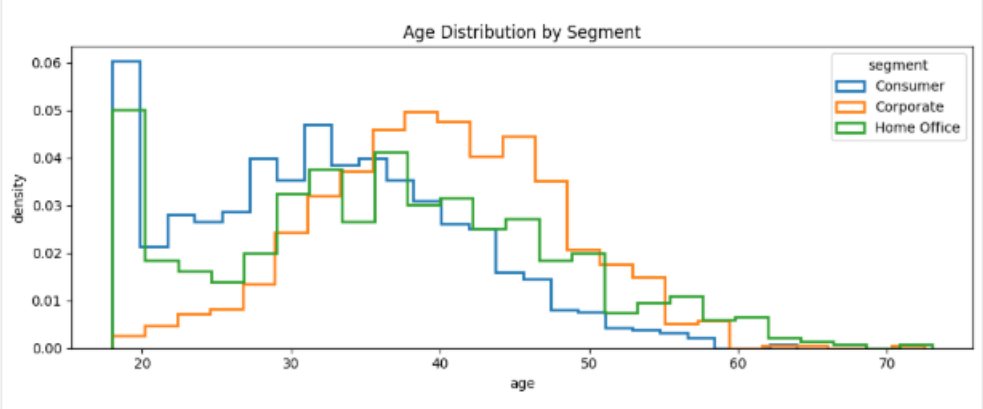
- Distribution of same variable across categories
- Overlapping colored histograms
- Quickly reveals group-level shifts

Interpretation

- Different peaks across segments → demographic variation
- Overlapping densities → shared ranges
- Few samples in one segment → interpret cautiously

HISTPLOT BY SEGMENT

```
sns.histplot(df, x="age", hue="segment", bins=25, element="step", stat="density", common_norm=False)
plt.title("Age Distribution by Segment")
# Segment-specific differences justify segment-based features
```



kdeplot — Smoothed Density Estimation

Kernel density estimate of continuous distribution

When to Use

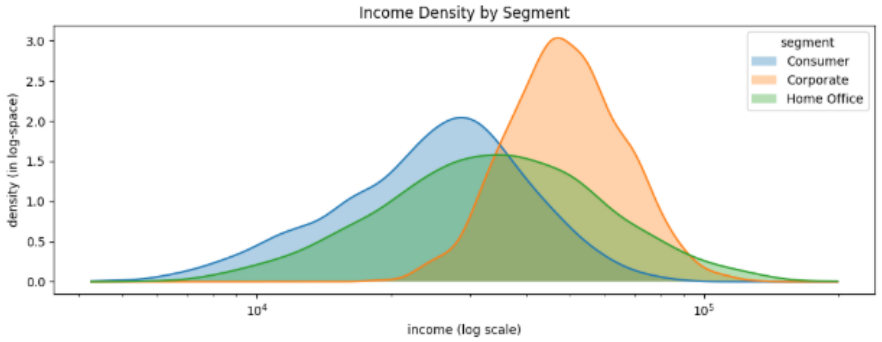
- Smooth view of distribution shape
- Comparing multiple distributions on same axis
- Optionally shaded (fill=True) for clarity

ML Implications

- Shifted density → premium vs regular customers
- Separated peaks → niche segments
- Suggests thresholds for derived features

KDEPLOT BY SEGMENT

```
sns.kdeplot(data=df, x="income", hue="segment", fill=True, common_norm=False, alpha=0.4)
plt.title("Income Density by Segment")
# Multi-peak densities may motivate mixture models
```



boxplot — Distribution and Outliers by Category

Median, quartiles, whiskers, and outliers

What boxplot Shows

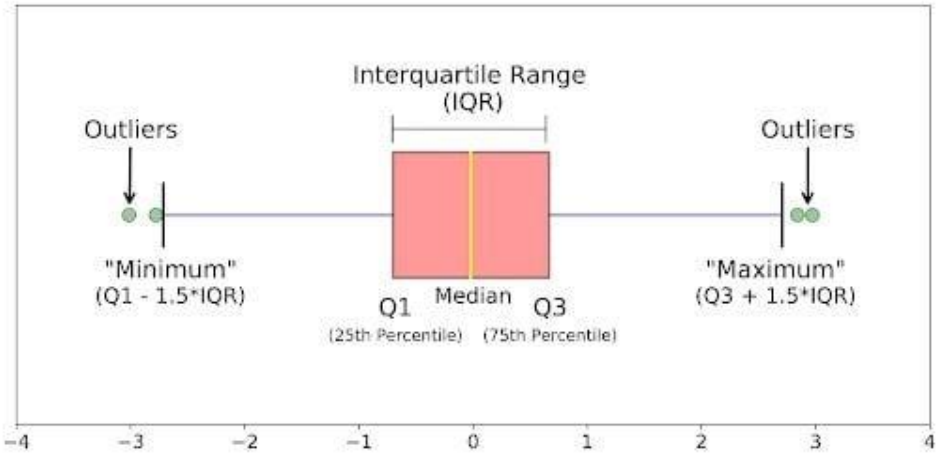
- Median, Q1, Q3, whiskers, outliers
- Side-by-side comparison across categories
- Compact view emphasizing spread and outliers

ML Implications

- Higher median → segment with typically higher values
- Long whiskers → more extremes in that segment
- Suggests segment-specific normalization

BOXPLOT BY SEGMENT

```
sns.boxplot(data=df, x="segment", y="income")
plt.title("Income by Segment — Boxplot")
# Identify groups where winsorization or scaling may differ
```



countplot — Category Frequency Visualization

Bar chart of counts for each category

What countplot Shows

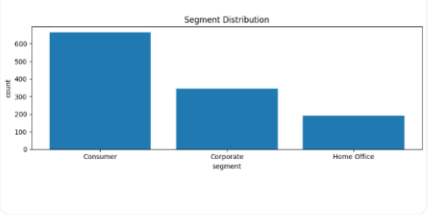
- Counts for each category of a categorical variable
- Optional hue for joint frequencies
- Quickly checks class imbalance

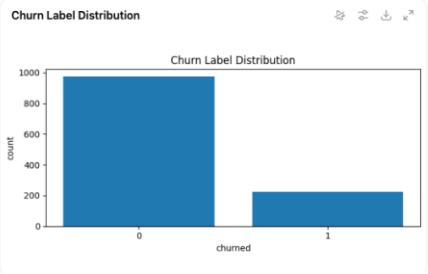
ML Implications

- Severe imbalance → stratified sampling, class weights
- Rare categories → grouping small categories
- Detects rare categories problematic for modeling

COUNTPLOT FOR CLASS IMBALANCE

```
sns.countplot(data=df, x="segment"); plt.title("Segment Distribution")
sns.countplot(data=df, x="churned"); plt.title("Churn Label Distribution")
# Churn imbalance suggests using resampling methods
```





Choosing the Right Seaborn Plot

Match plot type to variable types and goals

Decision Criteria

- Variable types (numeric vs categorical vs binary)
- Number of variables to visualize
- Focus: shape, central tendency, frequencies, relationships

Quick Reference

- Numeric distribution → histplot, kdeplot
- Numeric vs categorical → boxplot, violinplot
- Pure categorical → countplot
- Numeric vs numeric → scatterplot, relplot

PLOT SELECTION LOGIC

```
def recommend_plot(var_type_x, var_type_y=None):
    if var_type_y is None: return "histplot/kdeplot" if var_type_x == "numeric" else "countplot"
    return "boxplot" if var_type_y == "categorical" else "scatterplot"
```

DAY 32

Relationship Plots

Visualizing Multi-Variable Patterns

OBJECTIVES

- Use scatterplot for pairwise numeric relationships
- Use relplot for faceted multi-variable relationships
- Encode 3-4 variables via hue, style, size
- Recognize and mitigate overplotting
- Interpret segment-specific slopes and patterns

ACTIVITY

Create scatterplots encoding 3+ variables.

Optionally create relplot with facets.

ASSESSMENT

Submit visualization encoding ≥3 variables with interpretation.

scatterplot — Basic Numeric Relationship

Pairwise relationship as points

What It Shows

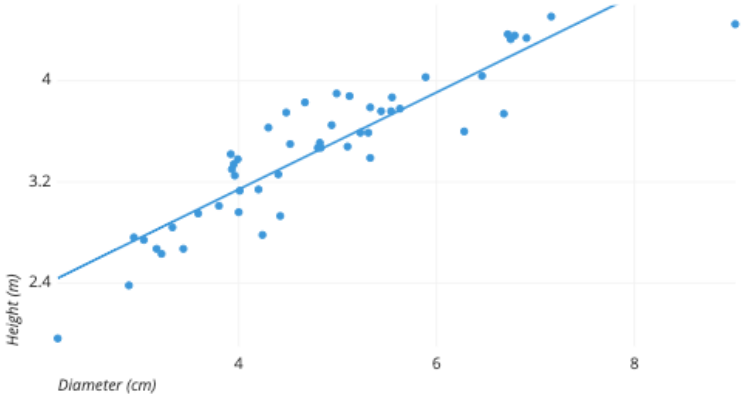
- Pairwise relationship between two numeric variables
- Reveals linearity, non-linearity, clusters, outliers
- Links features directly for potential interactions

ML Implications

- Linear relationship → supports linear models
- Outliers → robust models or transformations
- Heteroscedasticity → weighted regression

BASIC SCATTERPLOT

```
sns.scatterplot(data=df_rel, x="feature1", y="outcome", alpha=0.5)
plt.title("Outcome vs feature1 — Basic scatterplot")
# Upward trend suggests positive contribution to outcome
```



What Relationship Plots Don't Prove

Association ≠ Causation

Limitations

- Scatterplots show association, NOT causality
- Visual trends may be driven by unobserved confounders
- Overfitting to visual patterns is easy with small samples

Best Practice

- EDA generates hypotheses, not conclusions
- Features can be predictive without being causal
- Interpretability claims must be cautious

SPURIOUS CORRELATION EXAMPLE

```
df_spur = pd.DataFrame({"time_spent": 10+noise, "purchases": 5+noise*0.2})
sns.scatterplot(data=df_spur, x="time_spent", y="purchases")  # Correlated noise!
```

DAY 33

Heatmaps and Correlation Matrices

Feature Relationships and Multicollinearity

OBJECTIVES

- Compute correlation matrices with pandas
- Visualize correlations with sns.heatmap
- Interpret color scale (strength and direction)
- Detect feature clusters and multicollinearity
- Guide feature selection decisions

ACTIVITY

Generate and interpret correlation heatmap.

Identify multicollinearity candidates.

ASSESSMENT

What does a dark square indicate in a correlation heatmap?

Computing Correlation Matrices with pandas

Pairwise correlations between numeric columns

What df.corr() Does

- Computes pairwise correlations between numeric columns
- Default method: Pearson correlation
- Returns square matrix with values -1 to +1

When to Compute

- Before modeling: identify redundant features
- Explore linear relationships with target
- Understand structure of numeric feature space

CORRELATION MATRIX

```
corr_matrix = df_corr.corr()
print(corr_matrix)
# Diagonal = 1.0; off-diagonal shows pairwise correlations
# High absolute values indicate strong linear associations
```

	feature1	feature2	income	outcome
feature1	1.000	0.72	0.10	0.65
feature2	0.720	1.00	0.05	0.58
income	0.100	0.05	1.00	0.12
outcome	0.650	0.58	0.12	1.00

Basic Correlation Heatmap with sns.heatmap

Color-coded representation of correlation matrix

What Heatmap Shows

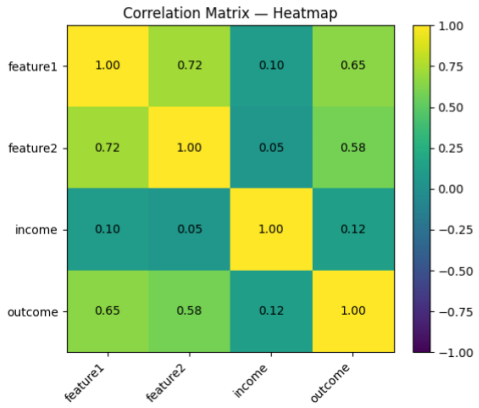
- Color indicates strength and direction
- Dark red: strong positive (+1)
- Dark blue: strong negative (-1)
- Light: weak or no linear relationship

ML Implications

- Strongly correlated predictors → multicollinearity
- Strong correlation with target → good linear candidates
- Feature clusters → potential redundancy

HEATMAP WITH ANNOTATIONS

```
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Matrix — Heatmap")
# annot=True shows actual values; cmap="coolwarm" diverging
```



ML Impact — Multicollinearity Detection

Using heatmaps to guide feature selection

Detection

- Identify pairs/groups with |corr| > 0.9
- Inflates variance of coefficient estimates
- Trees less affected but impacts feature importance

Actions

- Drop one feature from each pair
- Use dimensionality reduction (PCA)
- Apply regularization (Ridge, Lasso)

FINDING HIGH CORRELATION PAIRS

```
high_corr = []
for i in range(len(cols)):
    for j in range(i+1, len(cols)):
        if abs(corr_matrix.iloc[i,j]) > 0.9:
            high_corr.append((cols[i], cols[j], corr_matrix.iloc[i,j]))
```

DAY 34

Pattern Discovery with Advanced Plots

Faceting, Grouped Plots, and Trend Overlays

OBJECTIVES

- Use faceting to reveal subgroup patterns
- Create grouped plots for category interactions
- Apply trend overlays (regression lines, smoothers)
- Identify visual clusters and anomalies
- Distinguish signal from noise

ACTIVITY

Find non-obvious pattern using faceting/grouping.

Document pattern and modeling implications.

ASSESSMENT

How can faceting reveal patterns invisible in aggregated plots?

Faceting — Concept and Purpose

Split data into subsets in separate subplots

What Faceting Is

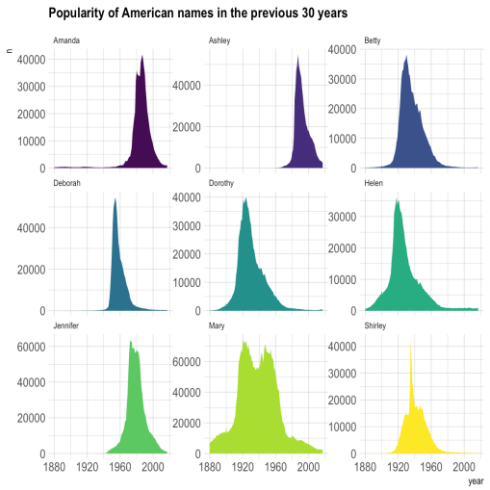
- Split data into subsets by category
- Plot each subset in separate subplot (row/col)
- Via relplot, displot, catplot in Seaborn

Why It Matters

- Reveals patterns that differ across categories
- Avoids overplotting by separating subgroups
- If patterns differ by facet, global model may be insufficient

FACETED RELPLOT

```
g = sns.relplot(data=df_pat, x="time", y="value", col="season", hue="segment",
                kind="scatter", alpha=0.4, height=3)
g.fig.suptitle("Value vs Time — Faceted by Season", y=1.05)
```



Grouped Plots — Category Interactions

Visualize distributions across category combinations

Concept

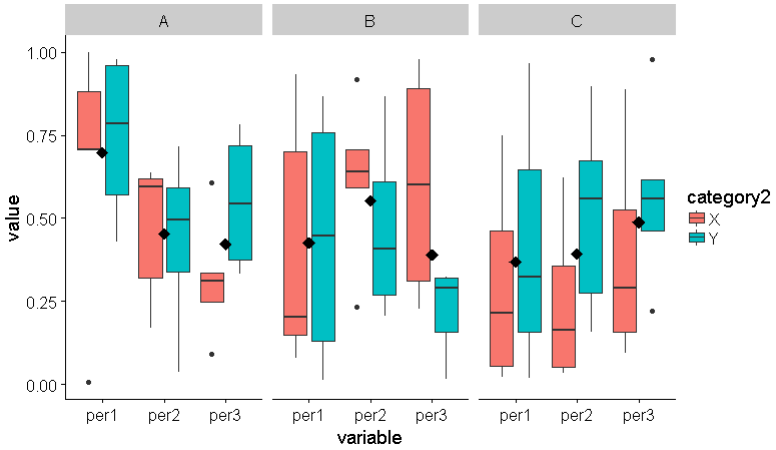
- Show numeric distributions across combinations of categories
- Grouped boxplots, grouped bar charts
- Highlight interactions between categories

ML Implications

- Differences in median/spread across buckets → evolving behavior
- Segment differences within bucket → interactions
- Suggests time bucket, season, segment features

GROUPED BOXPLOT

```
df_group["bucket"] = pd.cut(df_group["time"], bins=5, labels=["T1","T2","T3","T4","T5"])
sns.boxplot(data=df_group, x="bucket", y="value", hue="segment")
plt.title("Value by Time Bucket and Segment")
```



Trend Overlays — Regression Lines

Visually summarize relationships

Purpose

- Visually summarize relationship between variables
- Distinguish global trends from noise
- regplot (axes-level) or lmplot (figure-level)

Interpretation

- Red line approximates average trend
- If residuals vary by group, single line may mislead
- Supports inclusion of time terms in models

REGPLOT TREND OVERLAY

```
sns.regplot(data=df_pat, x="time", y="value", scatter_kws={"alpha": 0.3}, line_kws={"color": "red"})
plt.title("Value vs Time — Regression Trend Overlay")
```

Group-Specific Trends (lmplot)

Different regression lines per category

Why Group-Specific

- Capture different relationships per category
- Direct visual comparison of slopes and intercepts
- Show segment-specific effects

ML Implications

- Different slopes → segment × feature interactions
- Seasonal differences → seasonal features
- Encourage inclusion of interaction terms

LMPLOT BY GROUP

```
g = sns.lmplot(data=df_pat, x="time", y="value", hue="segment", col="season",
               scatter_kws={"alpha": 0.3}, height=3)
g.fig.suptitle("Grouped Trends by Segment and Season", y=1.05)
```

Identifying Visual Clusters

Groups of points closer to each other than others

What Clusters Show

- Natural subgroups in feature space
- Suggest latent segments
- Distinct blobs = meaningful groups

ML Implications

- Motivate clustering algorithms
- Suggest non-linear decision boundaries
- Overlapping clusters = imperfect separability

VISUAL CLUSTERS

```
cluster1 = np.random.normal([0,0], 1.0, (200,2))
cluster2 = np.random.normal([5,5], 1.0, (200,2))
df_clusters = pd.DataFrame(np.vstack([cluster1, cluster2]), columns=["x","y"])
sns.scatterplot(data=df_clusters, x="x", y="y")
```

DAY 35

EDA Mini-Project

Full Visual Story with ML Recommendations

OBJECTIVES

- Structure a complete EDA report
- Select meaningful visualizations for coherent story
- Link visual patterns to model/feature decisions
- Incorporate ethical and fairness considerations
- Produce actionable insights

ACTIVITY

Conduct full EDA on realistic dataset.

Write 3-5 key insights with ML recommendations.

ASSESSMENT

Graded EDA report with plots, interpretations, recommendations.

Structuring a Full EDA Report

Recommended sections for comprehensive analysis

Report Structure

1. Data overview (shape, types, missingness)
2. Univariate distributions (numeric, categorical)
3. Bivariate relationships (scatterplots, boxplots)
4. Multivariate patterns (facets, grouped, heatmaps)

First Steps

- Check shape, dtypes, missing values
- Understanding missingness guides imputation
- Variable types guide all visual choices

DATA OVERVIEW

```
print("Shape:", df_proj.shape, "\nDtypes:\n", df_proj.dtypes)
missing = df_proj.isna().sum().sort_values(ascending=False)
print("Missing:\n", missing)
```

Selecting Meaningful Visualizations

Choose high-value plots aligned with questions

Guidelines

- Limited set of high-value plots, not dozens
- Cover: distributions, relationships, groups, correlations
- Align each plot with specific question

Plot Selection

- Numeric: histplot, kdeplot, scatterplot, heatmap
- Categorical: countplot, boxplot, facets
- Each plot should teach something actionable

IDENTIFYING COLUMN TYPES

```
numeric_cols = df_proj.select_dtypes(include=["number"]).columns
cat_cols = df_proj.select_dtypes(include=["object", "category"]).columns
print("Numeric:", numeric_cols.tolist(), "\nCategorical:", cat_cols.tolist())
```

Univariate and Bivariate Visuals

Systematic visualization for each variable type

Step Sequence

- Histograms + KDEs for key numeric features
- Countplots for categorical features
- Boxplots for target vs category relationships

What to Record

- For each: skewness, outliers, group differences
- Flag variables needing transformation
- Identify potential group-level interactions

LOOPING THROUGH FEATURES

```
for col in numeric_cols[:4]:
    sns.histplot(df_proj[col].dropna(), bins=30, kde=True); plt.title(f"{col}")
    sns.boxplot(x=cat_cols[0], y=col, data=df_proj); plt.show()
```

DAY 35 · MINI-PROJECT

Correlation and Faceted Plots

Understanding feature structure and subgroup behavior

Correlation Heatmap

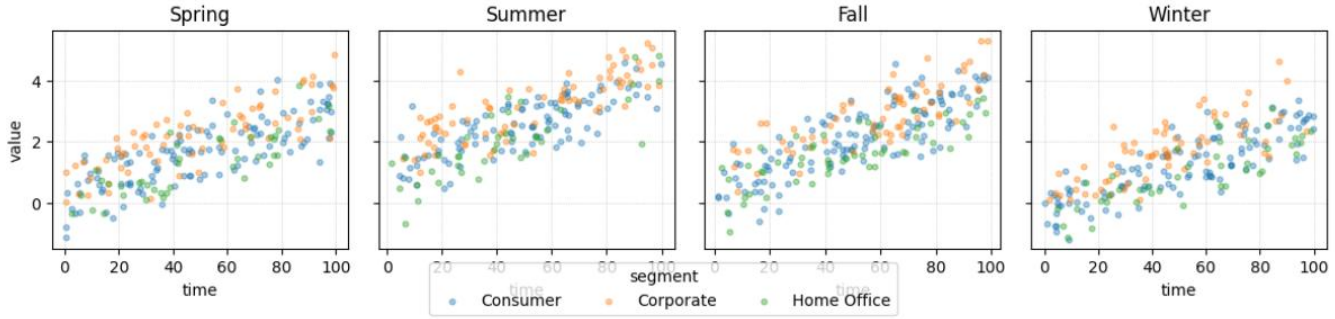
- Understand correlation structure
- Identify predictors correlated with target
- Find clusters for potential feature reduction

Faceted Relationships

- Reveal segment-specific relationships
- Check if relationships differ across categories
- Guide interaction terms or separate models

HEATMAP AND RELPLOT

```
sns.heatmap(df_proj[numeric_cols].corr(), cmap="coolwarm", vmin=-1, vmax=1)
g = sns.relplot(data=df_proj, x=numeric_cols[0], y=numeric_cols[1],
                hue=cat_cols[0], col=cat_cols[0], kind="scatter")
```



Linking Visuals to Modeling Decisions

From patterns to concrete actions

Visual → Modeling Link

- Heavy skew → apply log transforms
- Non-linear patterns → trees or polynomial features
- Strong correlations → drop or combine features
- Group differences → interactions or segment models

Key Principle

- EDA most valuable when it leads to explicit decisions
- Each insight should suggest an action
- Document rationale for each decision

MODELING ACTIONS LIST

```
actions = ["Apply transformations to skewed features", "Use robust scaling for outliers",
           "Drop highly correlated pairs", "Add interaction terms for group trends"]
```

Ethical and Fairness Considerations

Using EDA to detect potential bias

Patterns to Look For

- Systematic differences in target across sensitive groups
- Outlier behavior in specific demographic segments
- Features acting as proxies for protected attributes

Best Practice

- Use EDA to detect disparities, not justify discrimination
- Evaluate predictions and errors by group
- Document potential fairness concerns

IDENTIFYING SENSITIVE PROXIES

```
sensitive = [c for c in df_proj.columns if "gender" in c.lower() or "race" in c.lower() or "zip" in
c.lower()]
print("Potential sensitive/proxy columns:", sensitive)
```

DAY 35 · ACTIVITY

Full EDA Report

Comprehensive analysis with ML recommendations

Deliverables

1. Choose dataset with numeric + categorical + target
2. Univariate EDA: histplot, kdeplot, countplot
3. Relationship EDA: scatterplot, boxplot vs target
4. Correlation analysis: corr() + heatmap
5. Faceted/grouped plots for at least one segment
6. For each figure: 2-4 interpretation bullets + 1-2 ML implication bullets
7. Summary: 3-5 key insights → modeling decisions

Assessment Rubric

- Coverage: required visualizations included
- Interpretation quality: clear, correct bullets
- ML implications: explicitly articulated
- Visual clarity: appropriate color, hue, facets
- Ethical awareness: fairness concerns noted
- Coherence: narrative leads to modeling decisions